Stack Machines

Guillermo Polito













Goals

- Understand stack machines
- Stack operations
- Stack Machines for different code representations
  - trees
  - lists of instructions
- Real examples on Java, Ruby, Pharo…

Stack Machines

- Simple execution model where
  - Data is shared through a stack
  - Operations
    - consume values from the stack
    - produce values into the stack

Execution Example

```
(3 / (a + b)) . ceiling()
```



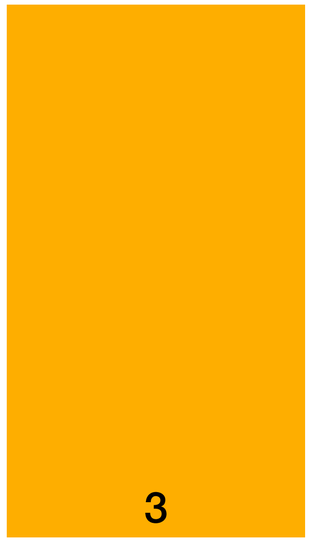
the stack

Execution Example

```
(3 / (a + b)) . ceiling()
```



Literal values
are pushed to the stack



the stack

Execution Example

```
(3 / (a + b)) . ceiling()
```



Variable values
are pushed to the stack



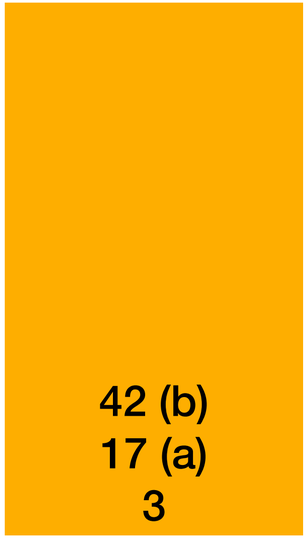
the stack

Execution Example

```
(3 / (a + b)) . ceiling()
```



Variable values
are pushed to the stack



the stack

Execution Example

```
(3 / (a + b)) . ceiling()
```



Arithmetic operations

1. **pop their operands**
2. **compute the result**
3. **push the result**



the stack

Execution Example

```
(3 / (a + b)) . ceiling()
```



Arithmetic operations

1. **pop their operands**
2. **compute the result**
3. **push the result**



the stack

Execution Example

```
(3 / (a + b)) . ceiling()
```



Method/function invocations

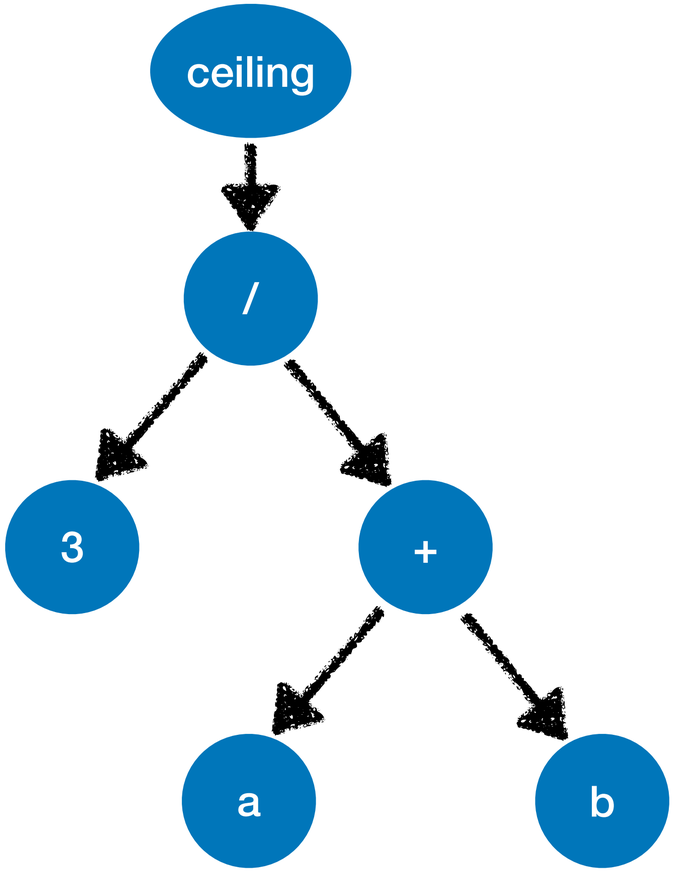
1. **pop the arguments**
2. **compute the result**
3. **push the result**



the stack

Tree Example

```
(3 / (a + b)) . ceiling()
```

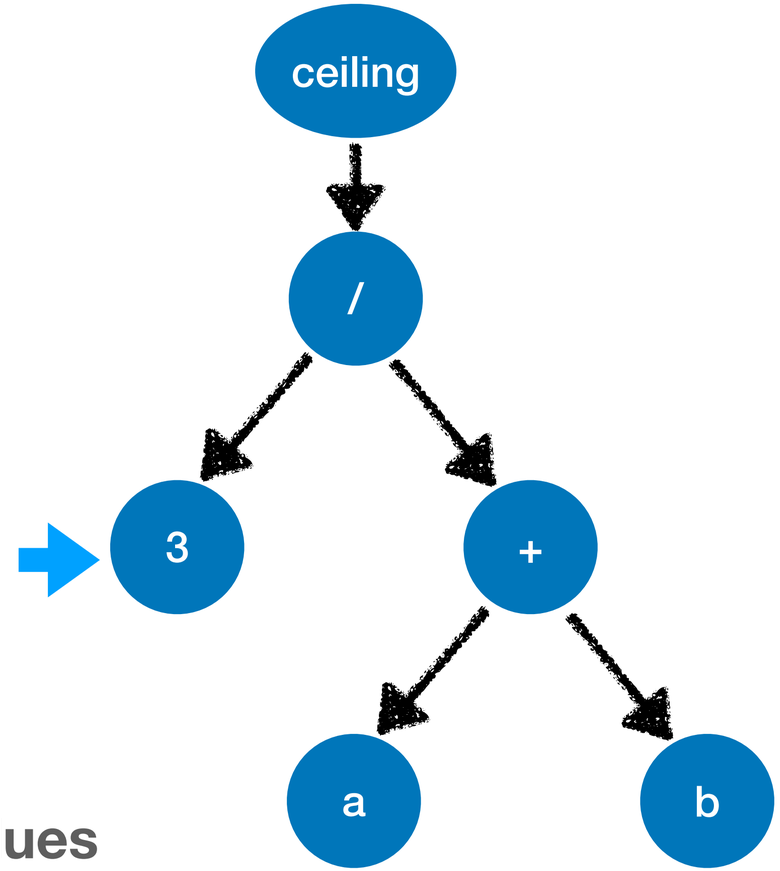




the stack

Tree Example: depth first post order

```
(3 / (a + b)) . ceiling()
```



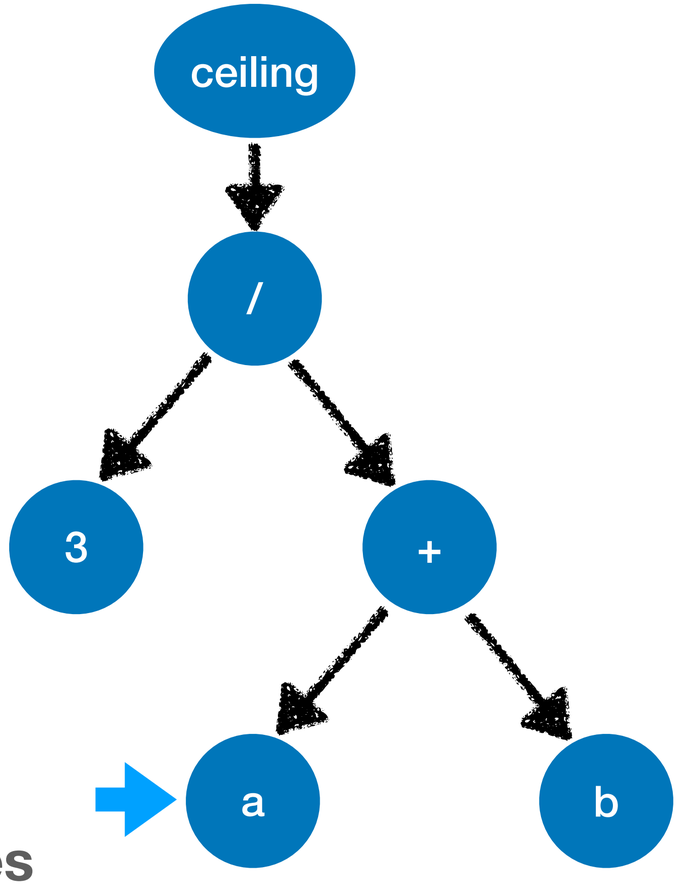


the stack

Literal values are pushed to the stack

Tree Example: depth first post order

```
(3 / (a + b)) . ceiling()
```



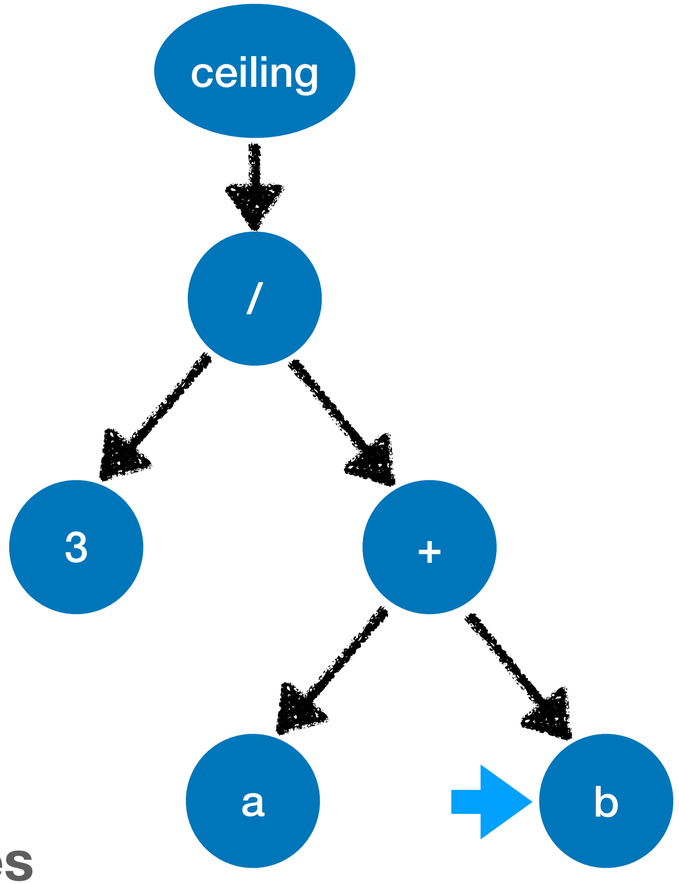


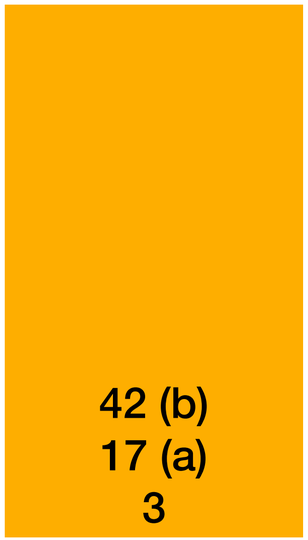
the stack

Variable values are pushed to the stack

Tree Example: depth first post order

```
(3 / (a + b)) . ceiling()
```



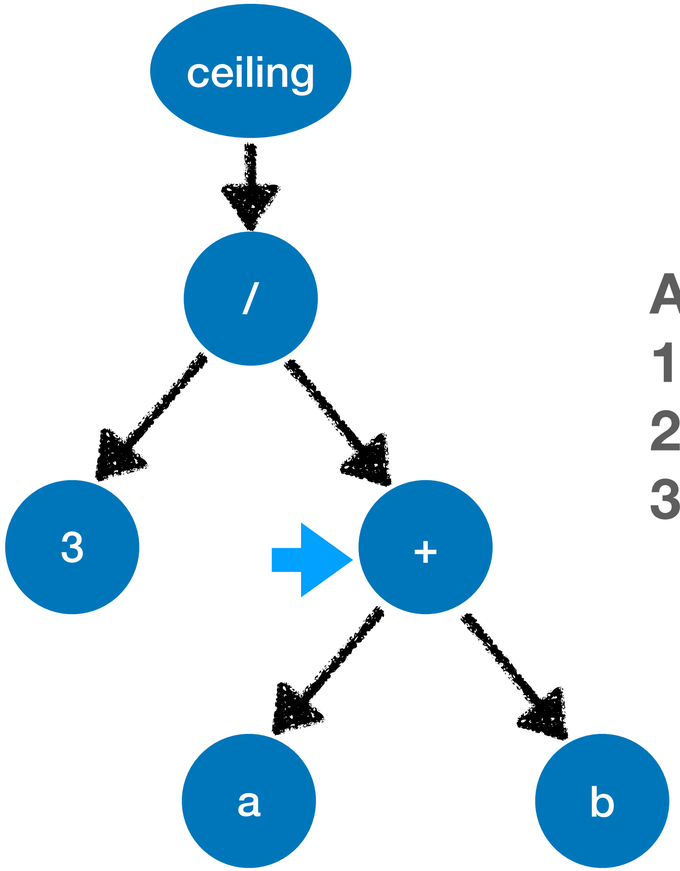


the stack

Variable values are pushed to the stack

Tree Example: depth first post order

```
(3 / (a + b)) . ceiling()
```



Arithmetic operations

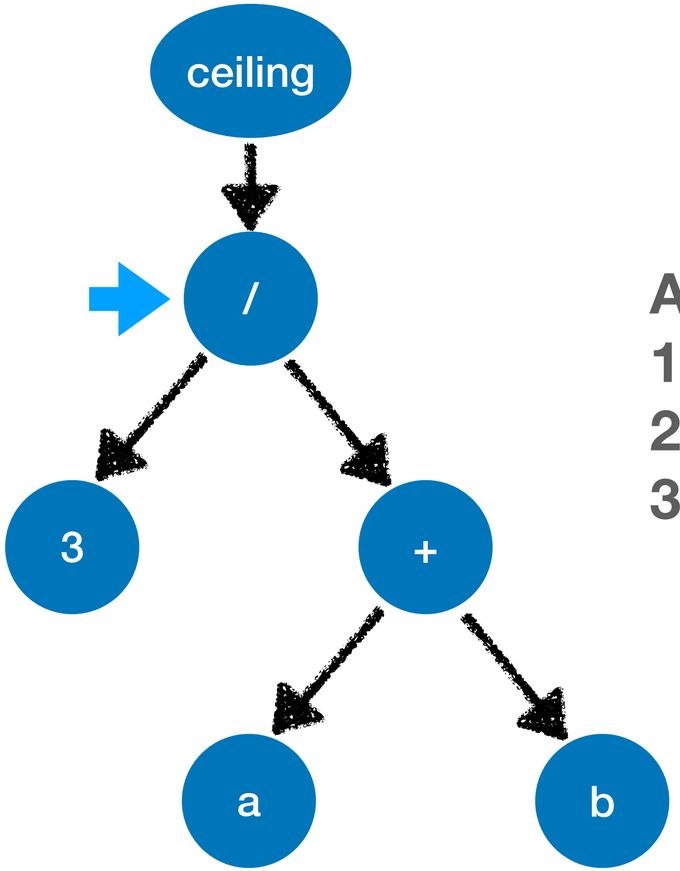
1. pop their operands
2. compute the result
3. push the result



the stack

Tree Example: depth first post order

```
(3 / (a + b)) . ceiling()
```



Arithmetic operations

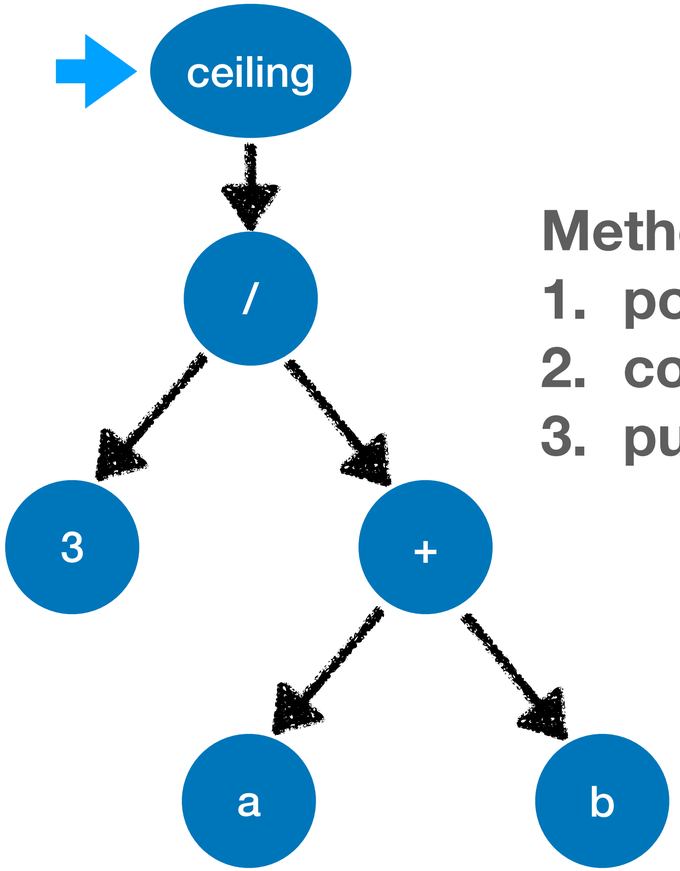
1. pop their operands
2. compute the result
3. push the result



the stack

Tree Example: depth first post order

```
(3 / (a + b)) . ceiling()
```



Method/function invocations

1. pop the arguments
2. compute the result
3. push the result



the stack

Stack-Based Bytecode Example

```
(3 / (a + b)) . ceiling()
```

```
push 3

push a

push b

send +

send /

send ceiling
```

- Compact linear representation
- Execution “falls” from one instruction to the next one

Stack-Based Bytecode

```
(3 / (a + b)) . ceiling()
```

```
push 3

push a

push b

send +

send /

send ceiling
```



the stack

Stack-Based Bytecode

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



the stack

Stack-Based Bytecode

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```

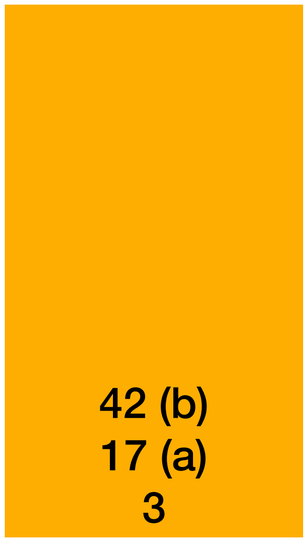


the stack

Stack-Based Bytecode

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



the stack

Stack-Based Bytecode

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



the stack

Stack-Based Bytecode

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



the stack

Stack-Based Bytecode

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



the stack

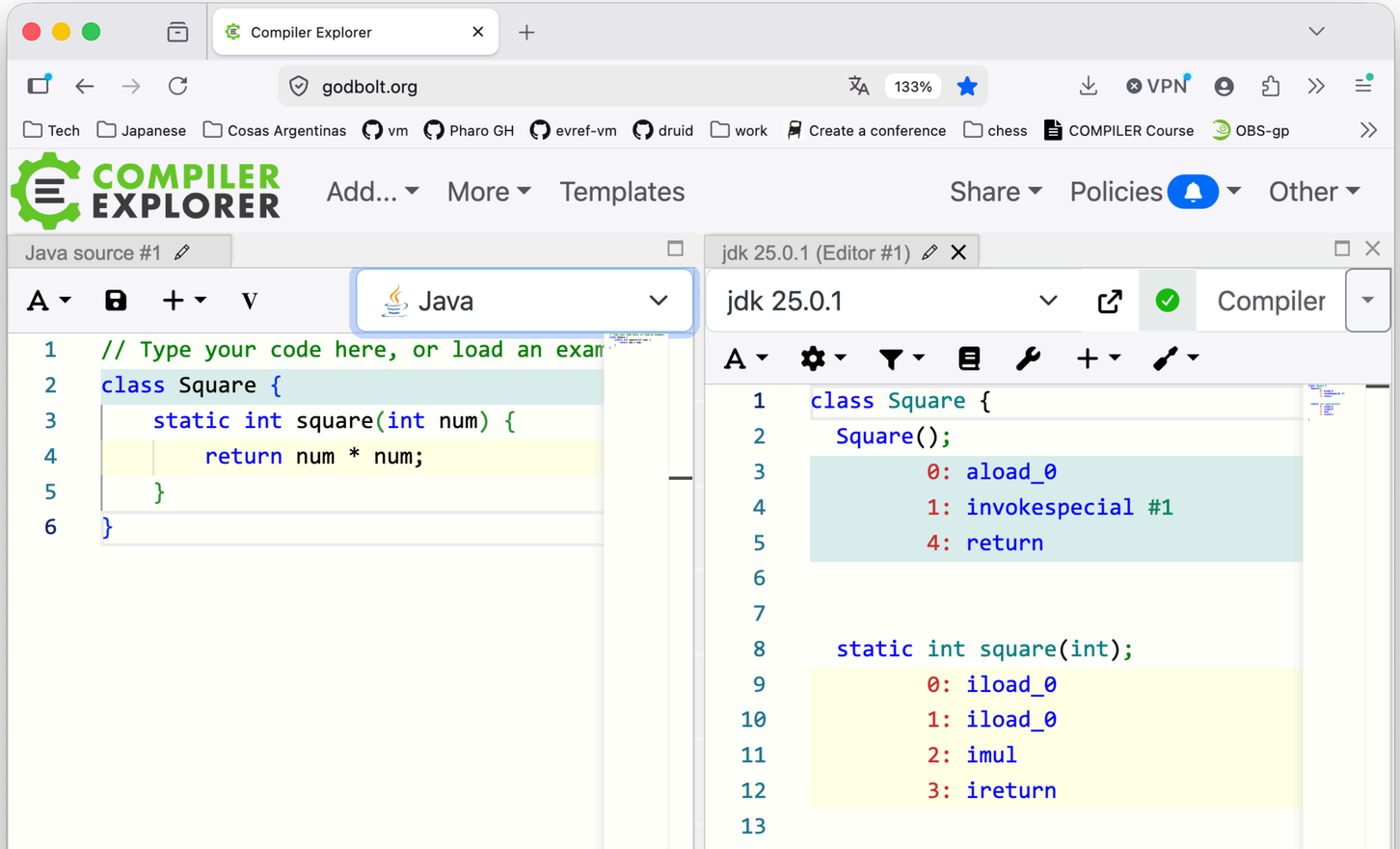
Special Stack Manipulation Instructions

- The design of stack machines require sometimes stack-specific instructions:

  - `pop` - pops the top of the stack
  - `fetch [n]` - pushes the nth element from the stack top
  - `dup` - pushes again the top of the stack, equivalent to `fetch 0`

Stack machines are popular

- They are simple to implement
- Examples:
  - JVM
  - Python
  - Pharo



Takeaways

- Stack machines are simple ways to implement languages
- They are very popular!

- Operations share values through an *operand* stack
  - They consume values (popping)
  - They produce values (pushing)
  - Special operations may be required by some implementations